

FRAMEWORK DE PERSISTENCIA

Bismach, Pablo
Ciancio, Facundo
Minelli, Nicolás

Ing. Cristian Ghilardi
V 1.0

¿Qué es un Framework?

Es un conjunto cohesivo de interfaces y clases que colaboran para proporcionar los servicios de la parte central e invariable de un subsistema lógico.

Tienen un fin específico y dan soporte para llevar a cabo determinadas operaciones.

Contiene clases concretas y especialmente abstractas que definen las interfaces a las que ajustarse, interacciones de objetos en las que participar, y otras invariantes.

Sin embargo, los frameworks no se encuentran completos, normalmente se requiere que el usuario del framework defina subclases de las clases del framework o lo complete mediante archivos de configuración.

Framework

- Existe una gran cantidad de frameworks disponibles en todo el mundo , cada uno con un propósito específico. Se utilizan para diversos aspectos de un sistema informático, existen frameworks de persistencia, de presentación, de reglas del negocio, etc.
- Aquellos frameworks que sobresalen utilizan tecnología de punta y conceptos novedosos. La única forma de que tengan el comportamiento deseado es con el uso de buenas prácticas, lo que implica un uso extensivo de patrones de diseño.

Objetivos de un esquema de persistencia

- Guardar objetos en un mecanismo persistente.
- Recuperar objetos de un mecanismo persistente.
- Administrar las operaciones persistentes con el fin de asegurar que sean atómicas.
- Permitir el uso de componentes y transacciones distribuidas.

En este caso se verá un conjunto de componentes (clases, objetos) que proporcionarán un servicio de persistencia al resto de la aplicación mediante el uso de un framework.

Un objetivo adicional es que nuestra aplicación no quede acoplada a este framework, es decir que el framework utilizado pueda cambiarse por otro framework sin afectar al experto.

¿Cómo se pueden lograr los objetivos de un esquema de persistencia?

- El impacto al ser incorporado en una aplicación debe ser mínimo, por lo tanto debe requerir la menor cantidad de modificaciones posibles a lo existente. Este aspecto no es menor y requiere de un buen diseño y el uso de patrones correctamente. Muchas veces debe resignarse algún aspecto, es decir, no lograr un gran diseño para que no tenga un impacto considerable, o que el impacto sea mayor pero nos permita tener una buena solución en el esquema.

¿Cómo se pueden lograr los objetivos de un esquema de persistencia?

- Las interfaces del esquema deben ser claras y concretas, facilitando su uso por parte de los desarrolladores.
- Debe proveer un servicio transparente al resto de la aplicación, esto en gran medida se logra respetando los aspectos analizados previamente, es decir, nace como una consecuencia de haber aplicado lo anterior.

Bases de Datos Relacionales

- La lógica de negocio no debería depender de dónde o cómo se guardan los datos ya que los datos pueden guardarse en diferentes esquemas de almacenamiento. El mecanismo de almacenamiento por excelencia son las bases de datos relacionales.

Bases de Datos Relacionales

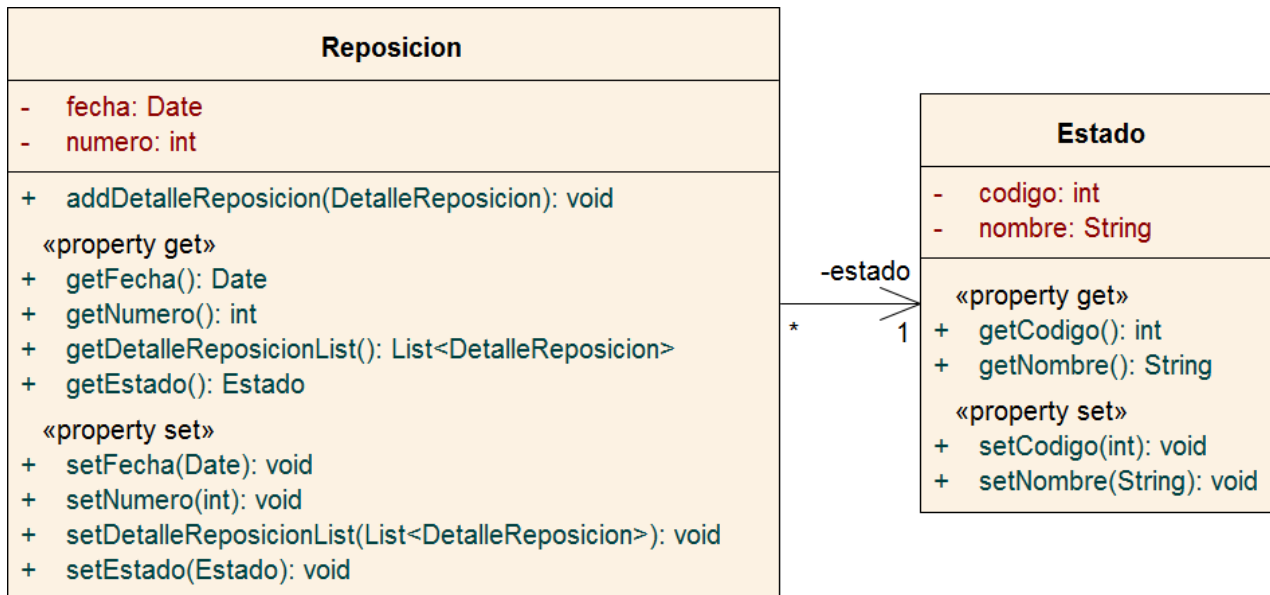
- Las bases de datos relacionales introducen el concepto de servicios de datos. Con ello permiten la administración de múltiples usuarios, bloqueos y acceso concurrente.
- Permiten definir estructuras eficientes para el almacenamiento de datos primitivos. Indexan su contenido y presentan una performance muy superior a otros mecanismos (archivos planos, documentos xml, bd orientadas a objetos).
- Establecieron el lenguaje de consultas SQL, de uso mundial.
- Presentan la gran desventaja de poseer una estructura muy diversa a aquella utilizada en las entidades de las reglas del negocio.

¿Qué es la persistencia?

- Permite que los datos de las aplicaciones sean persistentes en el tiempo. No se ven afectados por situaciones normales(apagado y encendido de equipos) ni anormales(cortes del suministro eléctrico).
- La necesidad de persistir no es nueva. Ha habido y van a haber múltiples formas de afrontar esta necesidad. El desafío consiste en conocer los límites de las distintas partes de un sistema y ser capaz de diferenciar capas y sus funcionalidades.

Entidades

- Representan datos del dominio de la solución. Deben encapsular sus datos y ser transparentes y reutilizables con distintos mecanismos de persistencia.
- Ejemplo: Reposición-Estado



Relaciones

- Todos los datos almacenados con cualquier mecanismo necesitan relacionarse. Es muy importante entender la diferencia entre el dato almacenado y la entidad que lo representa en las reglas del negocio.
- Mientras que entre entidades las relaciones se realizan a través de apuntadores entre los objetos, en las bases de datos relacionales se realizan mediante las claves primarias de las tuplas.

Identidad

- El estado de un objeto (especialmente de una entidad) está dado por los valores de sus atributos. Saber que un objeto es único depende de la regla del negocio.

Reposicion
- fecha: Date - numero: int
+ addDetalleReposicion(DetalleReposicion): void «property get» + getFecha(): Date + getNumero(): int + getDetalleReposicionList(): List<DetalleReposicion> + getEstado(): Estado «property set» + setFecha(Date): void + setNumero(int): void + setDetalleReposicionList(List<DetalleReposicion>): void + setEstado(Estado): void

- Una Reposicion es diferente de otra si tienen diferente número.

Identidad

- Para facilitar el funcionamiento, el esquema de persistencia debería poder determinar la unicidad de una entidad sin conocer la regla del negocio.
- El hecho de poseer identificadores que no sean compuestos mejora los tiempos de respuesta al realizar tareas de búsqueda en la base de datos y permite un manejo eficiente de la memoria caché.
- Los identificadores deberían ser únicos para cualquier entidad del sistema, no solo en aquellas del mismo tipo.
- La mayoría de los framework de persistencia existente requieren la existencia de un identificador único para cada entidad.

Identidad – OID

- Es necesario por propósitos de performance obtener algún tipo de referencia entre la entidad y el registro (o los registros) en la tabla correspondiente (o el mecanismo persistente que corresponda).
- Por esto se emplea el concepto de identificador único de objeto, a veces denominado OID. Es muy común implementarlo utilizando el patrón UUID, Unique Universal Identifier.
- Este patrón consiste en la generación de un identificador de 32 dígitos hexadecimales. Existen diversos mecanismos para generarlos, el más común es el de Leach-Salz.

Identidad – OID

- Las clases que se corresponden con el estereotipo de análisis “entidad” de nuestro modelo no tienen el atributo OID, necesario en el framework de persistencia. Para solucionar este problema, sin modificar las clases del negocio, podemos definir una clase “Entidad” de la que heredarán todas las clases persistentes.
- La clase “Entidad” no debe ser considerada como una clase persistente; se utiliza solamente para agregar a las clases hijas el comportamiento correspondiente al OID.

OID en nuestro ejemplo

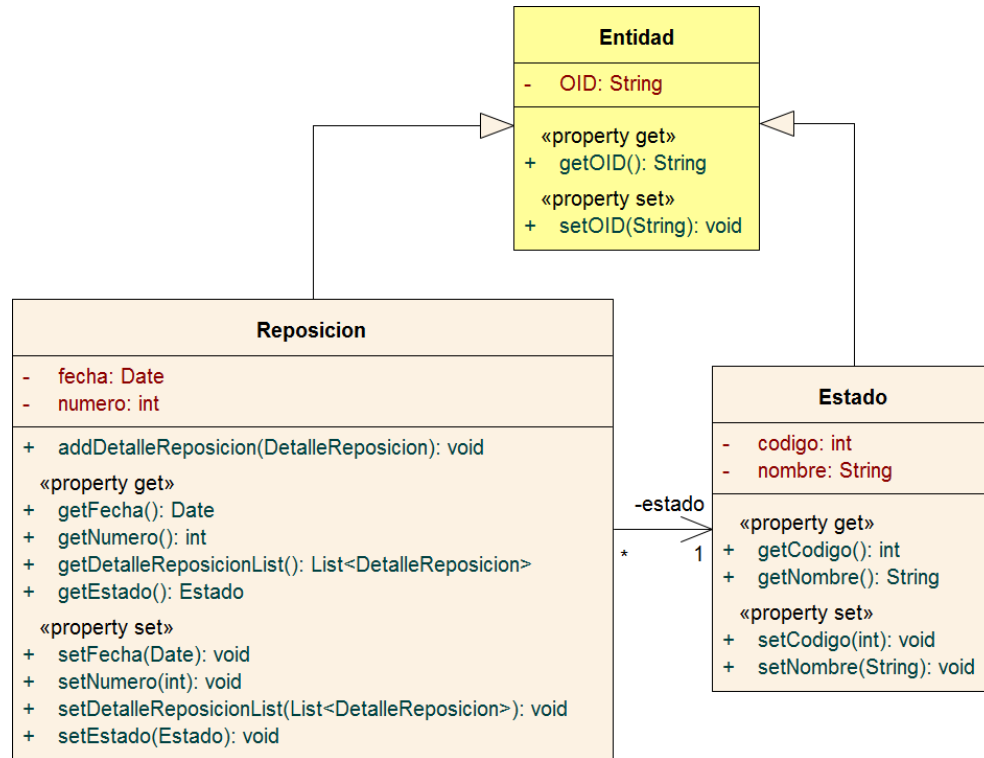


Tabla Reposicion

OID	numero	fecha	OIDEstado
4d9a885c-fcf8-11e5-870d-e8039a52bc18	3	2016-04-07	2db55a28-fcf8-11e5-870d-e8039a52bc18

Tabla Estado

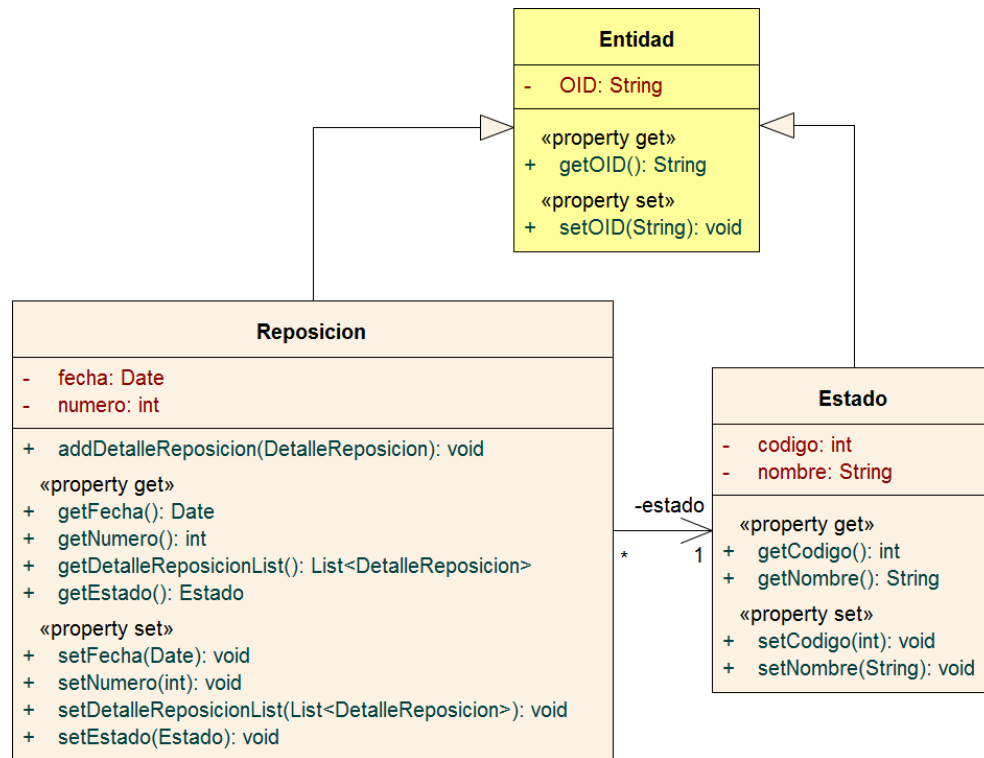
OID	codigo	nombre
2db55a28-fcf8-11e5-870d-e8039a52bc18	1	Creada

Framework de Persistencia – Hibernate

- Hibernate es una herramienta de Mapeo objeto-relacional (ORM) para la plataforma Java (y disponible también para .Net con el nombre de NHibernate) que facilita el mapeo de atributos entre una base de datos relacional tradicional y el modelo de objetos de una aplicación, mediante archivos declarativos (XML) o anotaciones en los beans de las entidades que permiten establecer estas relaciones.
- Hibernate es un framework que agiliza la relación entre la aplicación y la base de datos.

Mapeo

- Mediante el mapeo de entidades a estructuras relacionales se define la metodología (protocolo) para intercambiar datos entre los dos mecanismos.



Mapeo

Lista de atributos

Reposicion = OIDReposicion + numero + fecha + OIDEstado

Estado = OIDEstado + codigo + nombre

Mapeo en Hibernate

- El mapeo de objetos al utilizar Hibernate se realiza por lo general en un documento XML. Este lenguaje XML está centrado en Java, lo que significa que está construido alrededor de clases persistentes Java y no tablas.
- A continuación un ejemplo de las clases Reposicion y Estado mapeadas con XML para Hibernate:

Mapeo en Hibernate

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate...>
<hibernate-mapping>
  <class name="entidades.Estado" table="Estado">
    <id name="OID" type="string">
      <column name="OIDEstado"/>
      <generator class="uuid2"/>
    </id>
    <property name="codigo" type="int">
      <column name="codigo"/>
    </property>
    <property name="nombre" type="string">
      <column name="nombre"/>
    </property>
  </class>
</hibernate-mapping>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate...>
<hibernate-mapping>
  <class name="entidades.Reposicion" table="Reposicion">
    <id name="OID" type="string">
      <column name="OIDReposicion"/>
      <generator class="uuid2"/>
    </id>
    <property name="numero" type="int">
      <column name="numero"/>
    </property>
    <property name="fecha" type="date">
      <column name="fecha"/>
    </property>
    <many-to-one name="estado" column="OIDEstado"
      class="entidades.Estado" not-null="true"/>
    <bag name="detalleReposicionList" cascade="none"
      table="DetalleReposicion" inverse="false">
      <key column="OIDReposicion" not-null="true"/>
      <one-to-many class="entidades.DetalleReposicion"/>
    </bag>
  </class>
</hibernate-mapping>
```

Patrón Fachada

Fachada de Persistencia

Veamos un ejemplo de búsqueda de una entidad utilizando Hibernate.

```
HibernateUtil.getSession().beginTransaction();

Reposicion rep = (Reposicion) HibernateUtil.getSession().
    createCriteria(Reposicion.class).
    add(Restrictions.eq("numero", 3)).list().get(0);

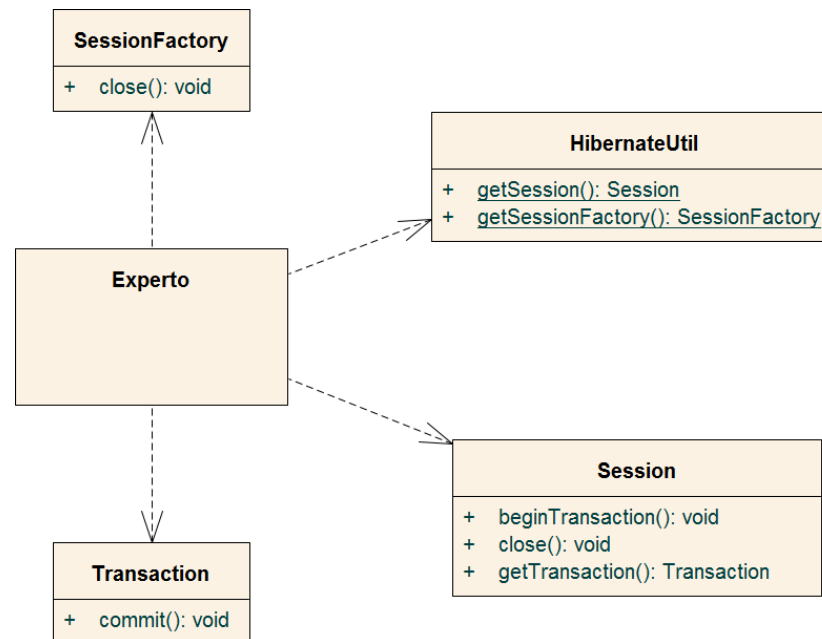
HibernateUtil.getSession().close();
```

Este es un ejemplo de búsqueda para la Reposicion con el numero 3 desde el experto usando Hibernate.

Patrón Fachada

Fachada de Persistencia

- Si realizáramos la búsqueda desde el Experto utilizando Hibernate este quedaría acoplado a las distintas clases que provee Hibernate para su funcionamiento.



Patrón Fachada

Fachada de Persistencia

- Esto no es aconsejable ya que si decidiéramos cambiar de framework el impacto de la modificación repercutiría directamente en el Experto.

Patrón Fachada

Fachada de Persistencia

- Para disminuir el acoplamiento entre el experto con las clases del framework de persistencia que vamos a utilizar el patrón Fachada (Facade).
- El patrón Fachada oculta la complejidad interna de un componente o conjunto de componentes, y provee un punto de acceso simplificado y estandarizado.

Patron Fachada

Fachada de Persistencia

Contexto/Problema:

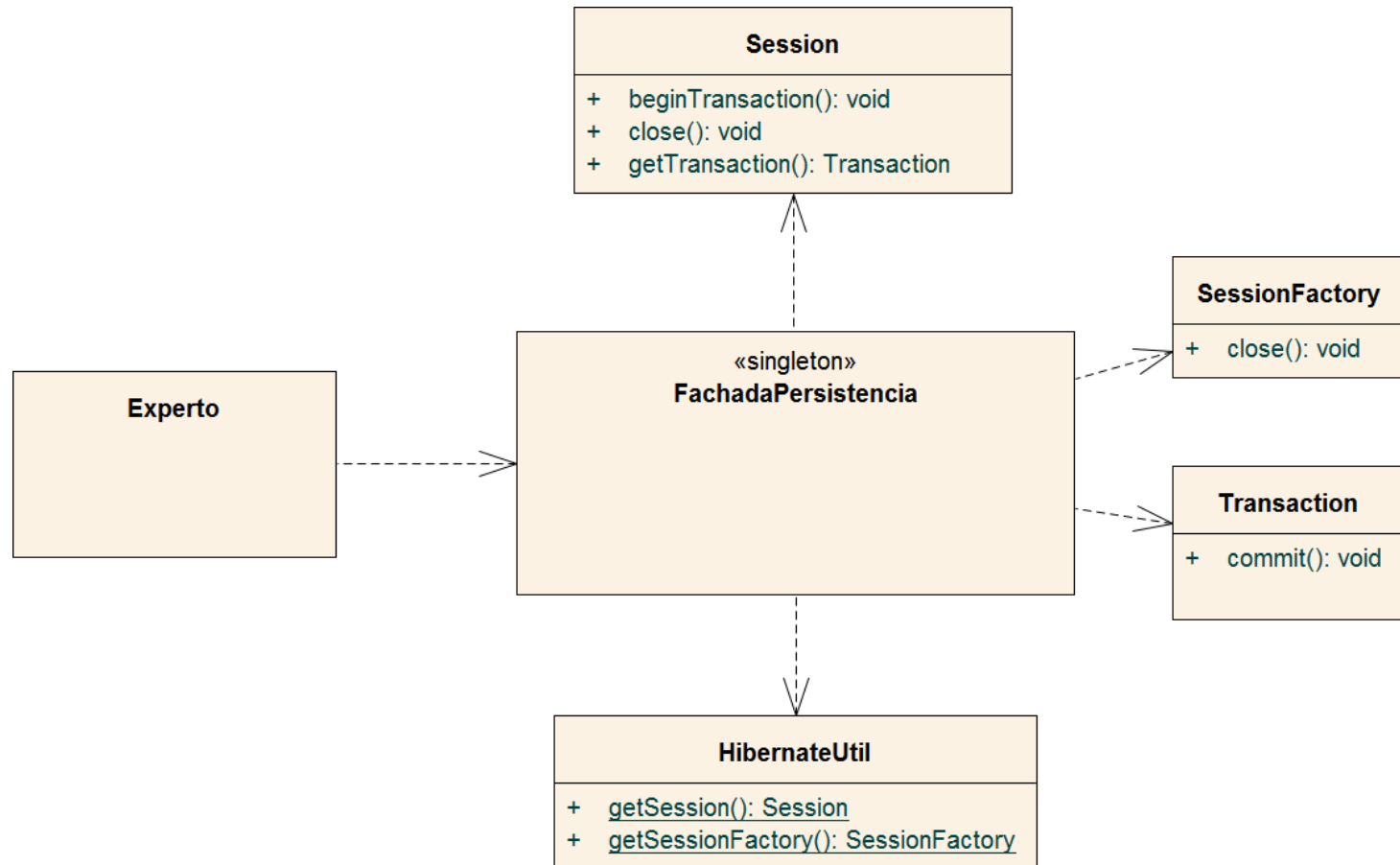
- Se requiere una interfaz común, unificada para un conjunto de implementaciones o interfaces dispares-como un subsistema-. Podría no ser conveniente acoplarlas con muchas cosas del subsistema, o la implementación del subsistema podría cambiar. ¿Qué hacemos?

Solución

- Defina un único punto de conexión con el subsistema -un objeto fachada que envuelve al subsistema-. Este objeto fachada presenta una única interfaz unificada y es responsable de colaborar con los componentes del subsistema.

Patrón Fachada

Fachada de Persistencia



Patrón Fachada

Fachada de Persistencia

Ventajas

- Oculta la complejidad del subsistema
- Simplifica las interfaces
- Simplifica el uso de las funcionalidades internas

Desventaja

- Esta fuertemente acoplado a las clases que utilicen la persistencia.

Debemos tener en cuenta que si cambia la fachada cambiarían todos nuestros expertos

Patrón Fachada

Fachada de Persistencia

- En nuestro caso la fachada será la encargada de intermediar entre el experto y el esquema de persistencia, con el fin de simplificar la conexión con el mismo y mejorar la cohesión del experto.

«singleton» FachadaPersistencia	
-	<u>instancia: FachadaPersistencia</u>
+	<u>getInstancia(): FachadaPersistencia</u>
+	buscar(String, String): List<Object>
+	guardar(Object): void

- Podemos implementar la Fachada de Persistencia como un Singleton.

Patrón Fachada

Fachada de Persistencia

- Ya que necesitamos un punto de acceso estandarizado, vamos a dotar a la fachada de dos métodos genéricos: `buscar()` y `guardar()`. Estos métodos contendrán en su interior la comunicación con el framework. De esta manera si decidiésemos cambiar de framework sólo tendríamos que modificar los métodos de la Fachada de Persistencia y no los Expertos.

Fachada de Persistencia

Criterio

- Necesitamos utilizar un lenguaje para poder realizar las búsquedas de nuestros objetos. En el caso de Hibernate se utiliza HQL.
- Si queremos que nuestro framework de persistencia sea intercambiable (por ejemplo por JPA) debemos tener en cuenta que este lenguaje debe tener una equivalencia en los posibles candidatos al reemplazo. La fachada debe convertir el lenguaje de consulta que utilicemos al lenguaje de consulta utilizado por el framework.

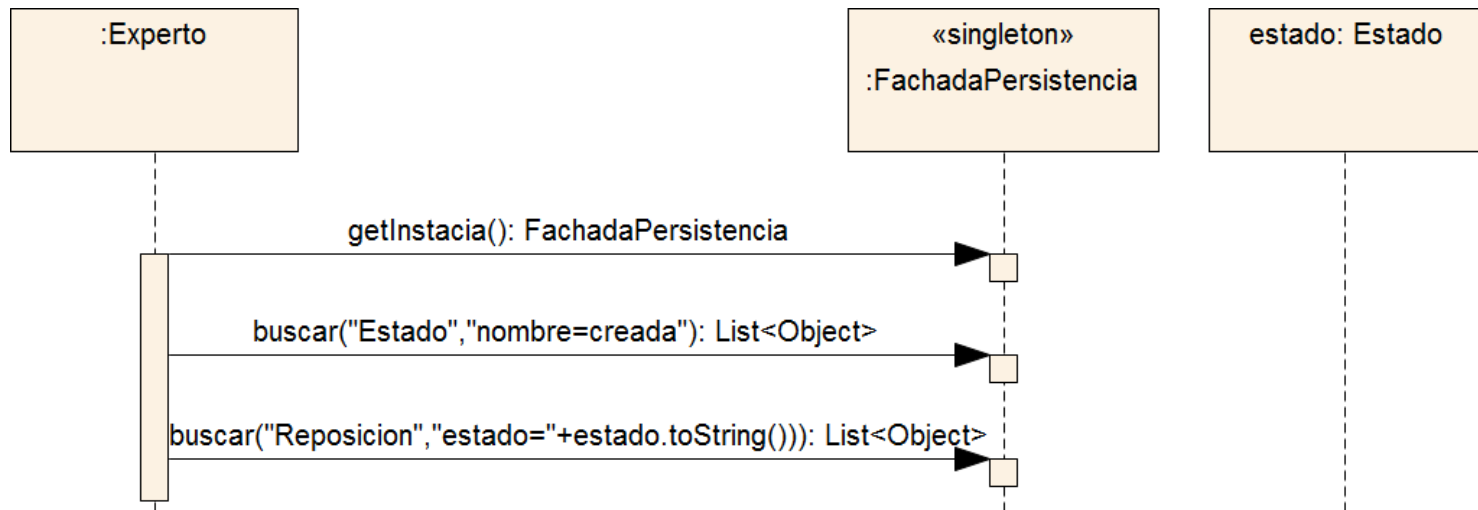
Fachada de Persistencia

Criterio

- Se busca estandarizar los parámetros de búsquedas de la fachada, de manera tal de que estos no sean propios del framework de persistencia que se está utilizando (por ejemplo, sería un error enviar como argumento el String con el HQL de la búsqueda del objeto).
- Para permitir cambiar de framework fácilmente, usaremos el mismo lenguaje de consulta de la primera parte de la materia. Este lenguaje es muy simple, pero nos asegura una implementación en varios framework existentes. (ver Documento "Busqueda con Indirección (Modelo de Comportamiento)"

Fachada de Persistencia

Criterio



- Habíamos señalado que el método `estado.toString()` no era implementable fácilmente.
- Para solucionar este problema para las búsquedas utilizaremos un DTO Criterio.

Fachada de Persistencia

Criterio

- **Atributo (String)**: Colocamos el nombre del atributo que vamos a comparar, dicho atributo debe pertenecer a la Clase sobre la que se buscara el objeto. El campo "Atributo" será de tipo String ya que solo vamos a guardar el nombre de dicho atributo.
- **Operador (String)**: Colocamos la operación que queremos realizar, será de tipo String como "Atributo" ya que también solo guardaremos el Operador.
- **Valor (Object)**: Colocamos el valor con el que el Atributo va a ser comparado, es decir, la restricción. Al hacerlo de tipo Object, nosotros podremos comparar utilizando por ejemplo: Integer, String, Boolean, o incluso una entidad de nuestro sistema.

Fachada de Persistencia

Criterio

DTOCriterio
- atributo: String - operacion: String - valor: Object
«property get» + getAtributo(): String + getOperacion(): String + getValor(): Object «property set» + setAtributo(String): void + setOperacion(String): void + setValor(Object): void

Fachada de Persistencia

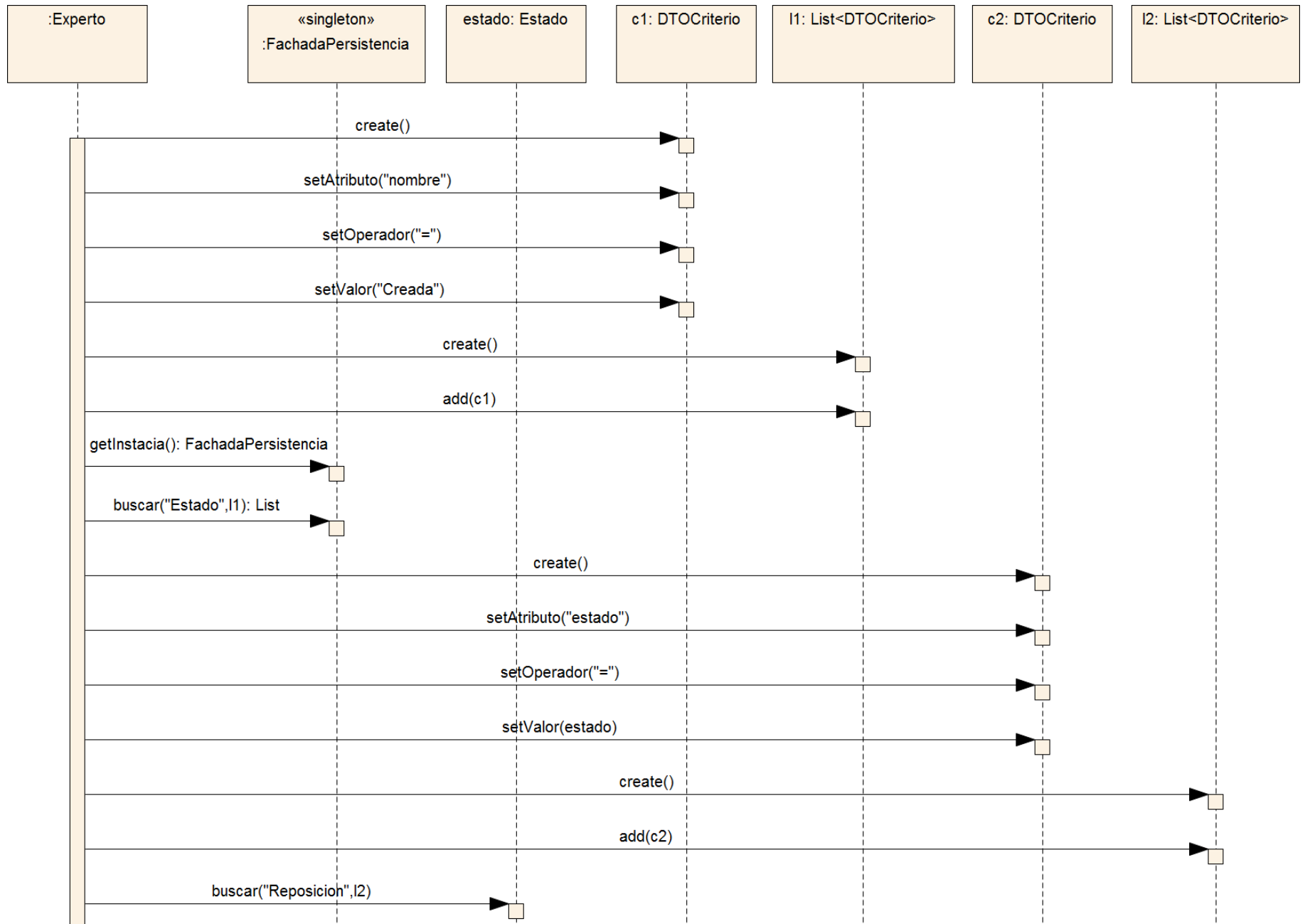
Criterio

De éste modo la Fachada nos ofrecería un método Buscar que tendrá dos parámetros:

- El primero será un String que contiene el nombre de la clase que deseamos buscar.
- Y el segundo será una lista de Criterios, ya que la búsqueda de un objeto puede estar sujeta a múltiples condiciones, y se creará un objeto Criterio por cada una de ellas.

Y un método guardar similar al visto con la indirección de persistencia.

«singleton» FachadaPersistencia	
-	<u>instancia: FachadaPersistencia</u>
+	<u>getInstancia(): FachadaPersistencia</u>
+	buscar(String, List<DTOCriterio>): List<Object>
+	guardar(Object): void



Transacciones

- Una transacción es un mecanismo que permite asegurar la ejecución atómica de un caso de uso determinado.
- El concepto atómico se refiere a la necesidad de ejecutar un caso de uso como una unidad, es decir, ejecutarlo completo ó no ejecutarlo en absoluto.
- Para que un caso de uso sea atómico deberíamos iniciar una transacción antes de comenzar un caso de uso y finalizarla al terminar.

Transacciones

Los frameworks de persistencia a partir de las características ofrecidas por las bases de datos ofrecen mecanismos para asegurar la atomicidad de los cambios realizados.

Dichas órdenes se denominan:

- **BeginTransaction**: inicia una transacción.
- **Commit**: implica confirmar todo lo realizado desde que comenzó la transacción.

Siguiendo nuestro razonamiento con respecto a la fachada deberíamos agregar un método `iniciarTransaccion()` y `finalizarTransaccion()`

Transacciones – Decorador

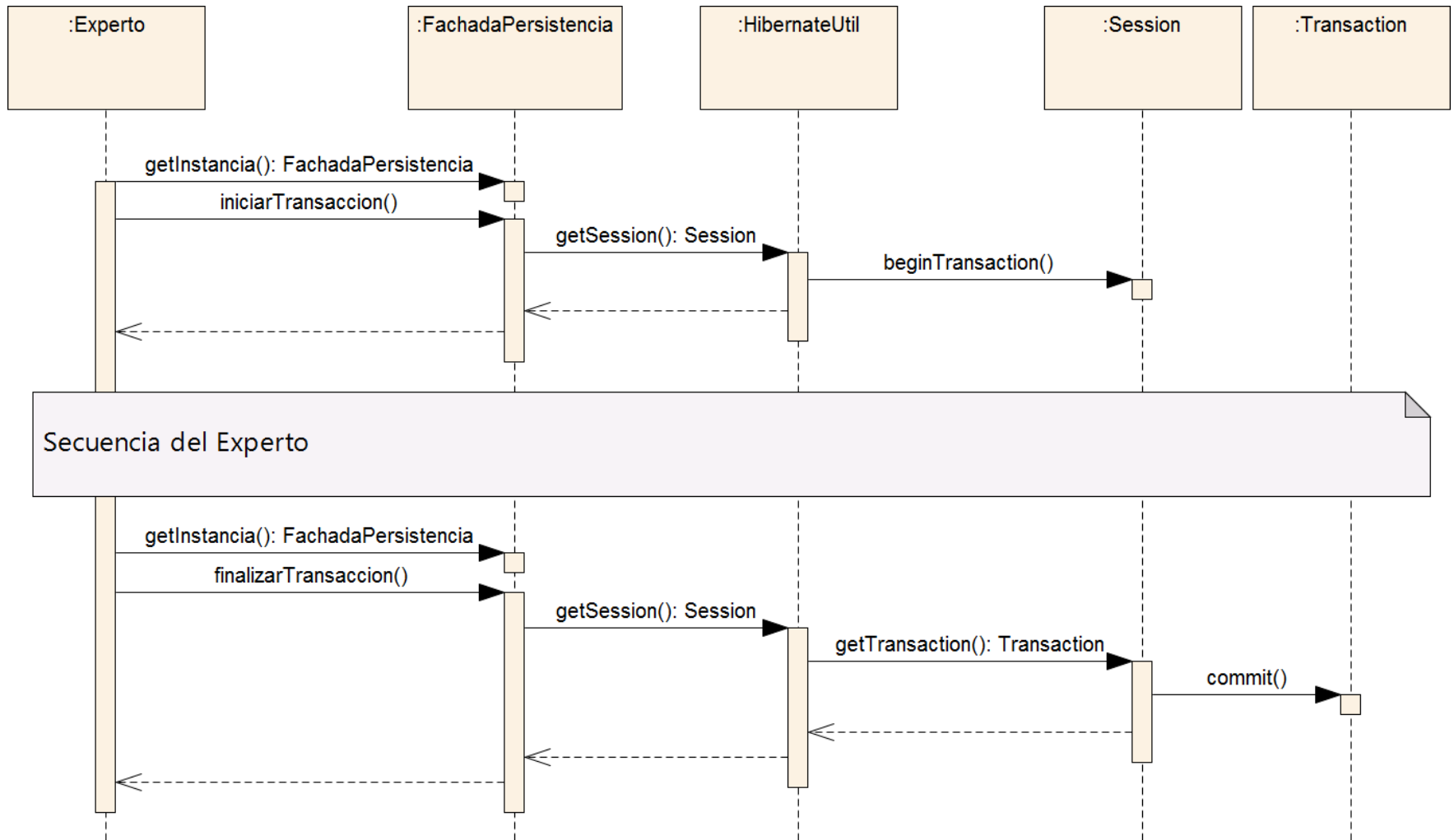
Vemos los métodos de la Fachada de Persistencia

«singleton» FachadaPersistencia	
-	<u>instancia: FachadaPersistencia</u>
+	<u>getInstancia(): FachadaPersistencia</u>
+	buscar(String, List<DTOCriterio>): List<Object>
+	guardar(Object): void
«transacciones»	
+	iniciarTransaccion(): void
+	finalizarTransaccion(): void

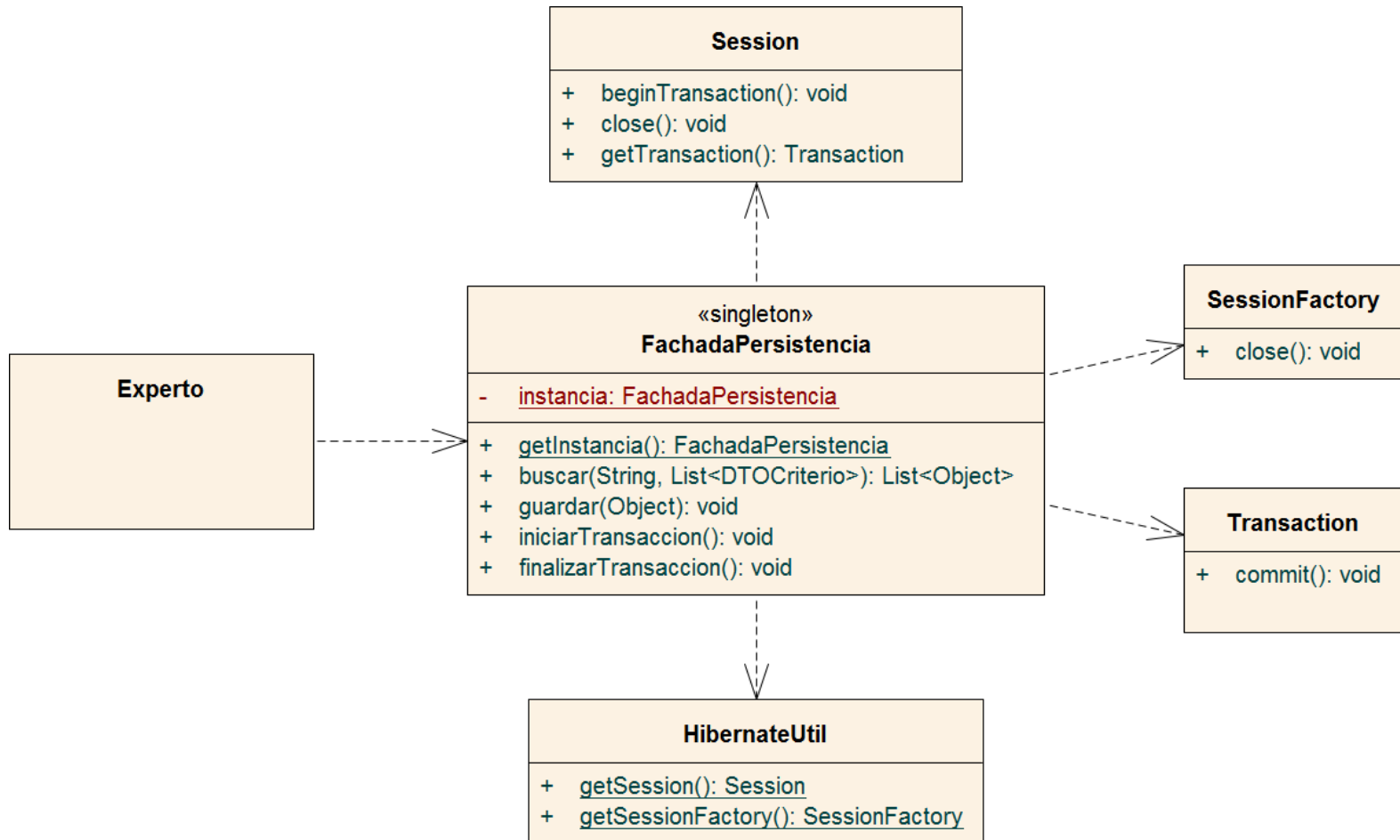
Podemos Observar que la fachada de persistencia posee los métodos comenzarTransaccion() y finalizarTransaccion().

Un ejemplo de uso con Hibernate sería el siguiente:

Transacciones



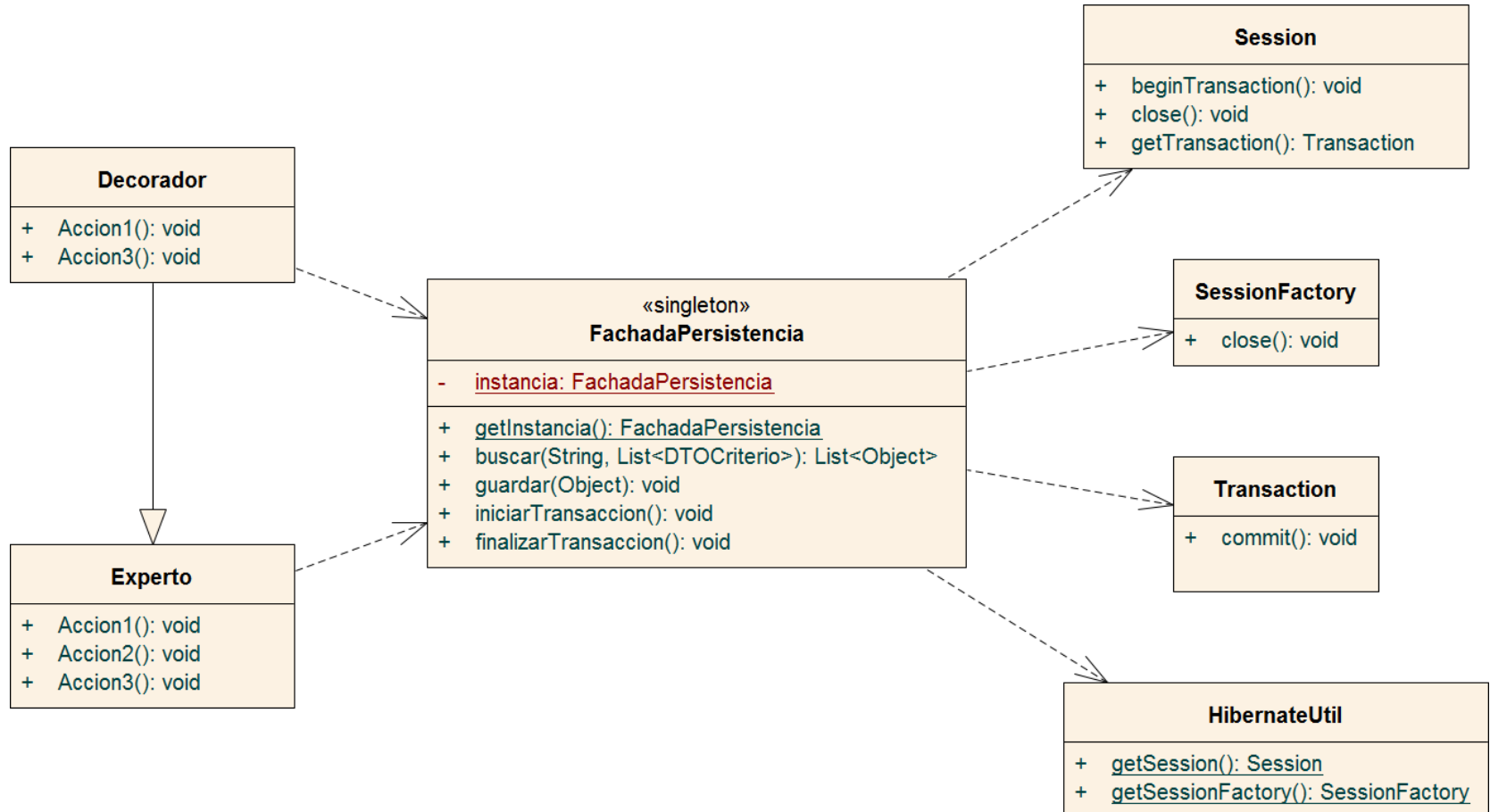
Transacciones



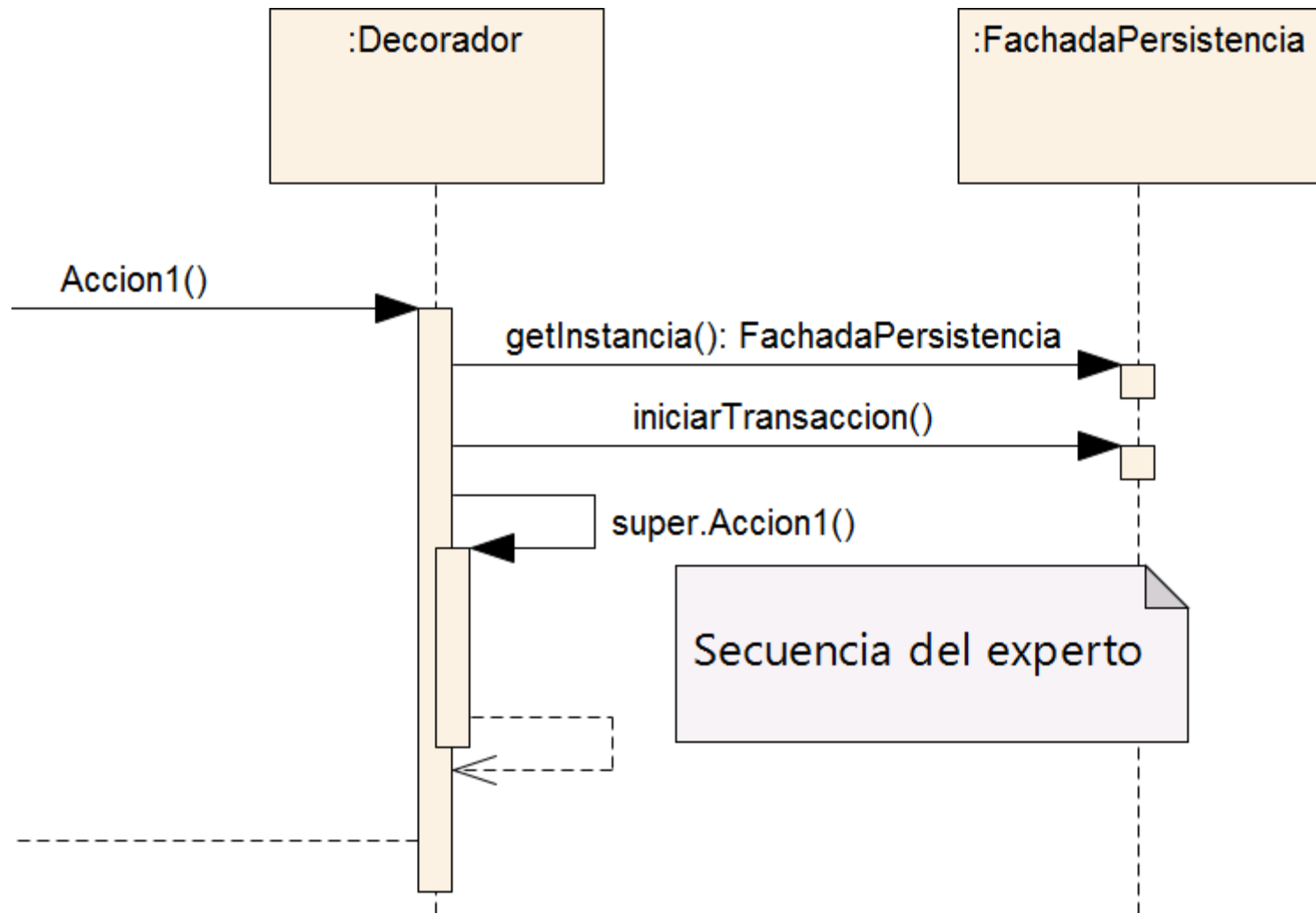
Transacciones

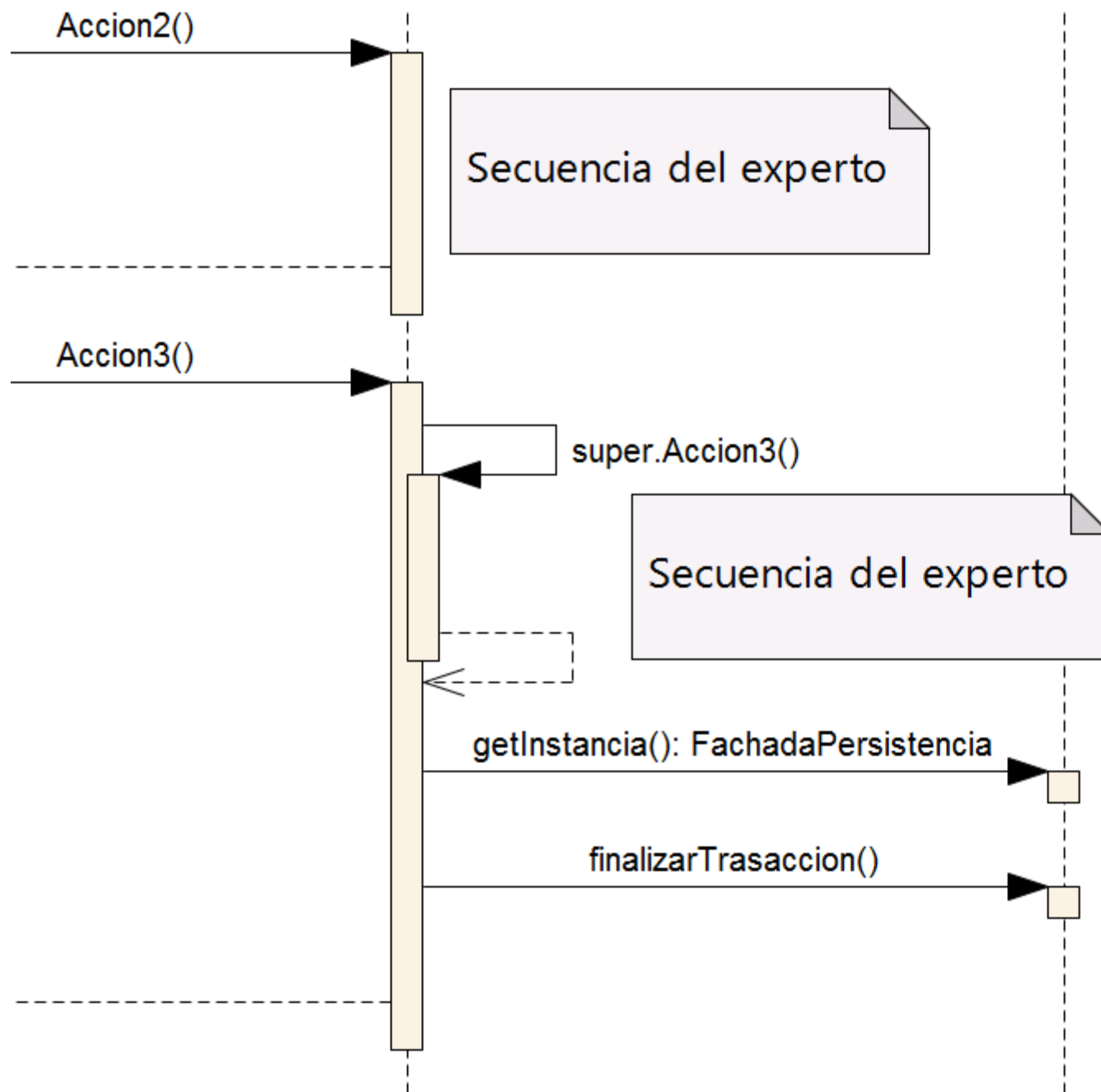
- En el ejemplo anterior el experto es el encargado de iniciar y terminar la transacción.
- Esto no sería recomendable, el experto pierde cohesión dado que además de conocer las reglas de negocio debe preocuparse por la atomicidad del caso de uso.
- El patrón Decorador permite rodear una tarea teniendo la oportunidad de realizar algo antes y/o después de la misma.
- Y es lo que justamente buscamos, iniciar la transacción en el inicio del CU y finalizarla al terminar el CU.
- Veamos cómo implementarlo:

Transacciones – Decorador



Transacciones





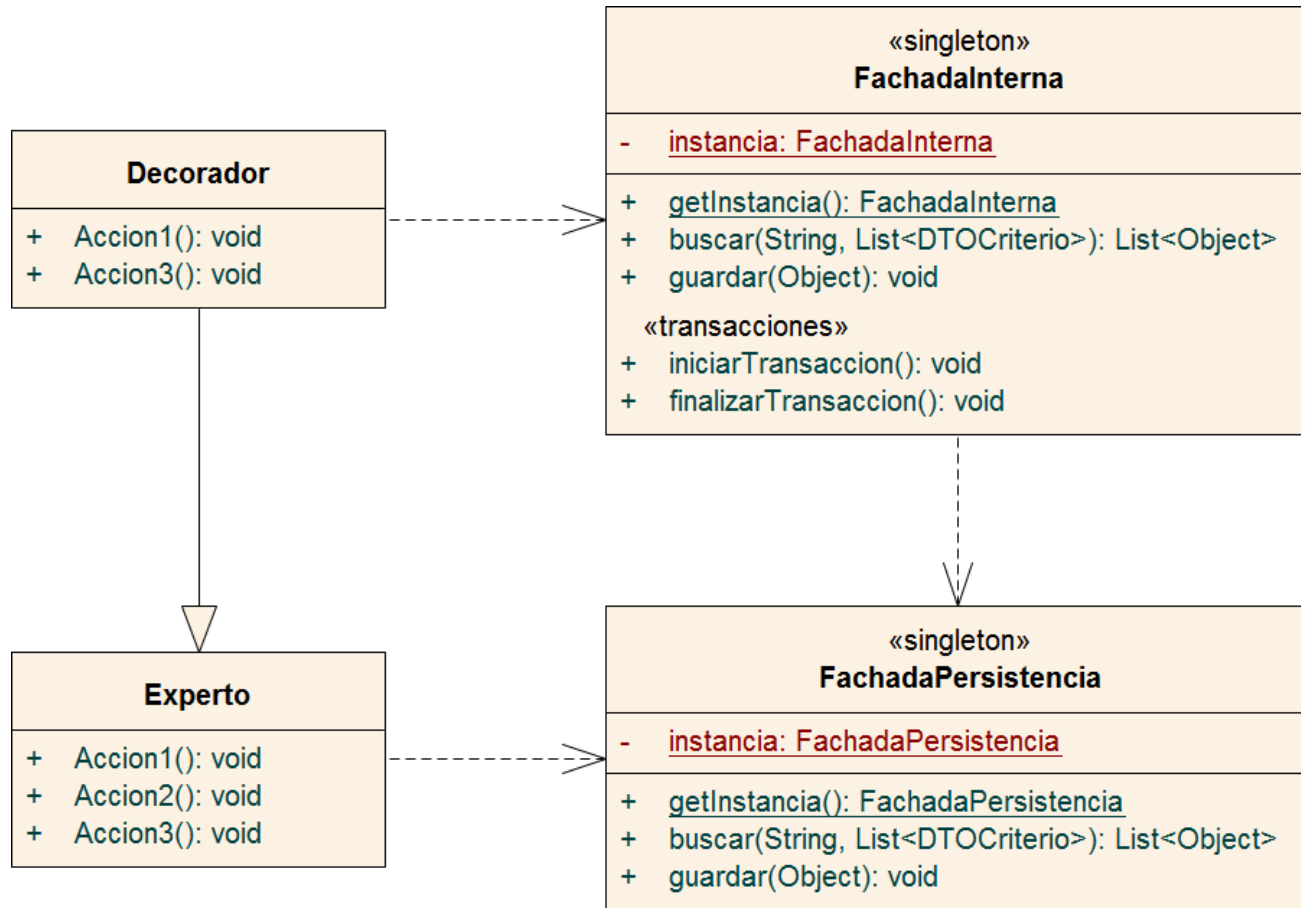
Transacciones – Decorador

```
public class Decorador extends Experto {  
  
    @Override  
    public void Accion1() {  
        FachadaPersistencia.getInstance().comenzarTransaccion();  
        super.Accion1();  
    }  
  
    @Override  
    public void Accion3() {  
        super.Accion3();  
        FachadaPersistencia.getInstance().finalizarTransaccion();  
    }  
}
```

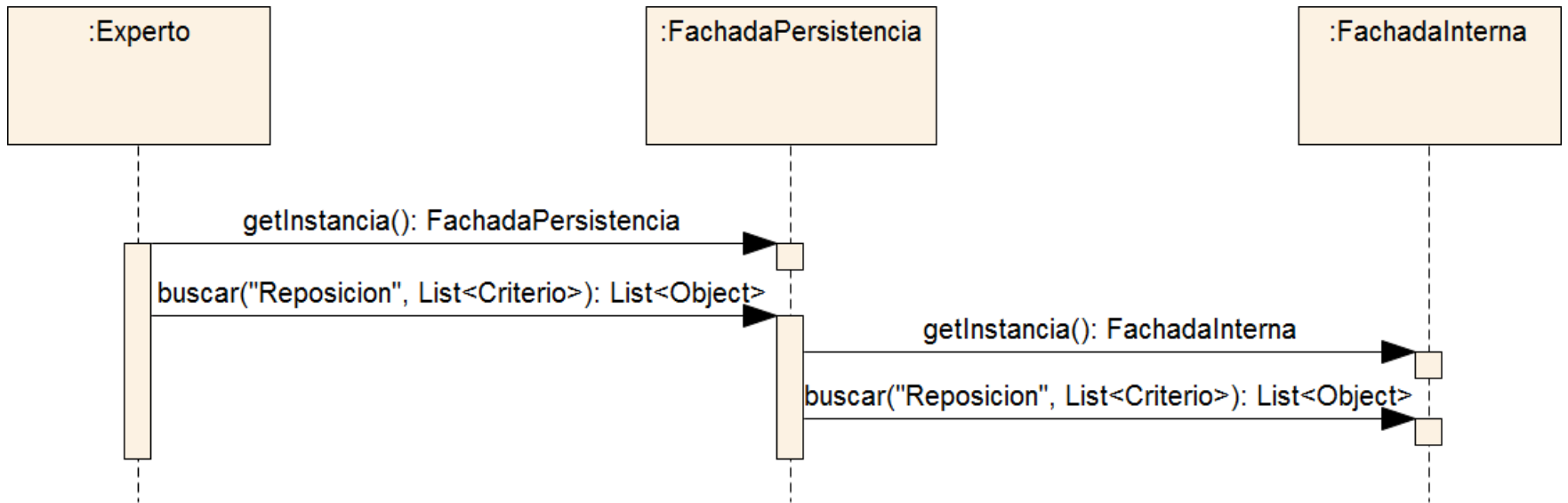
Transacciones – Decorador

- El problema surge en que el experto tiene una dependencia con la Fachada de Persistencia y, por lo tanto, puede acceder a los métodos relacionados con el manejo de transacciones.
- Podríamos pensar entonces en utilizar dos Fachadas para dar solución a esto.
- Una que preste servicios hacia el “exterior” y otra que preste servicios hacia el “interior”. La primera simplemente deriva las solicitudes a la segunda.
- De esta manera poseemos una fachada de negocio disponible a los expertos y una fachada interna disponible al decorador para el uso de transacciones. Así evitamos el acceso indeseado de los expertos a los métodos relacionado con las transacciones.

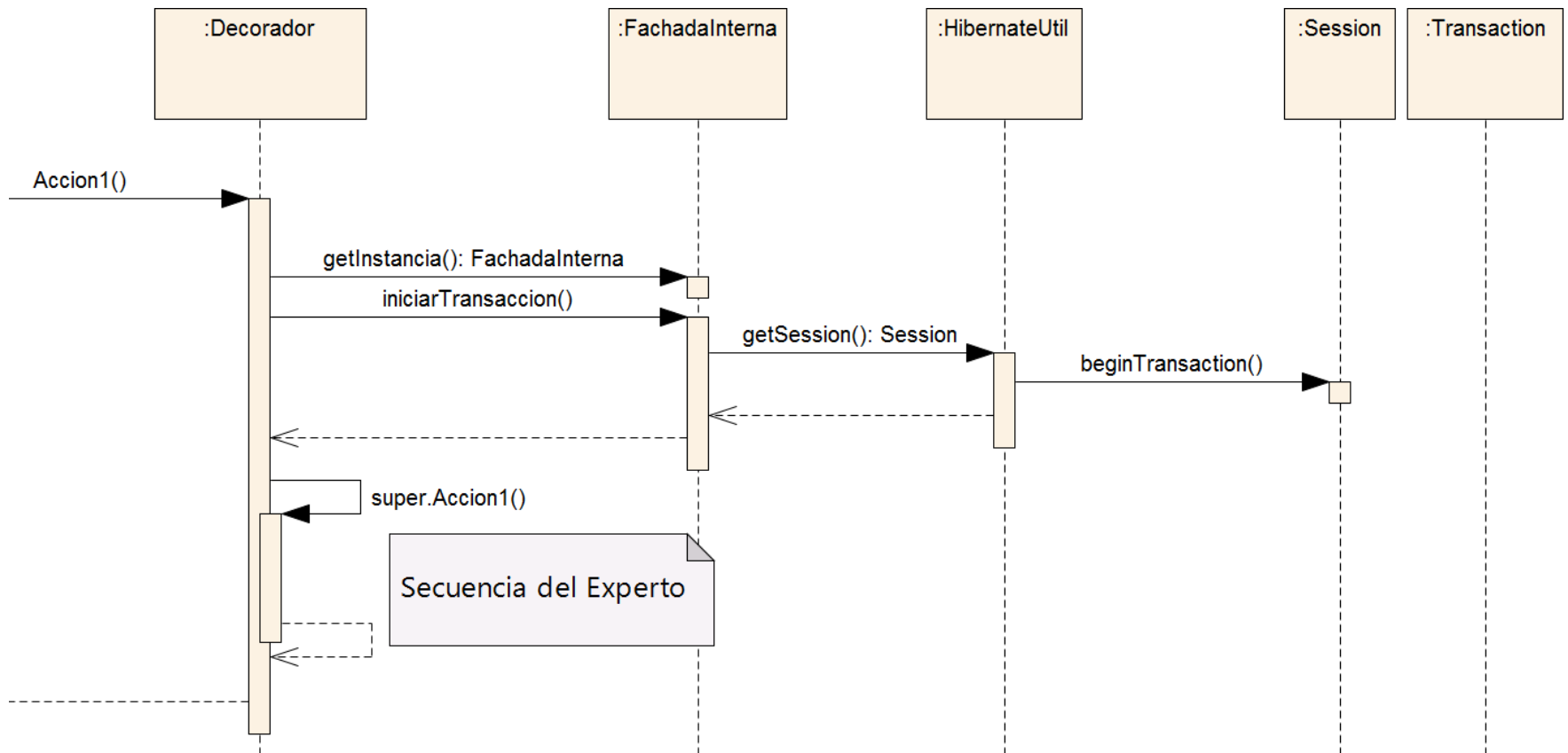
Transacciones – Decorador



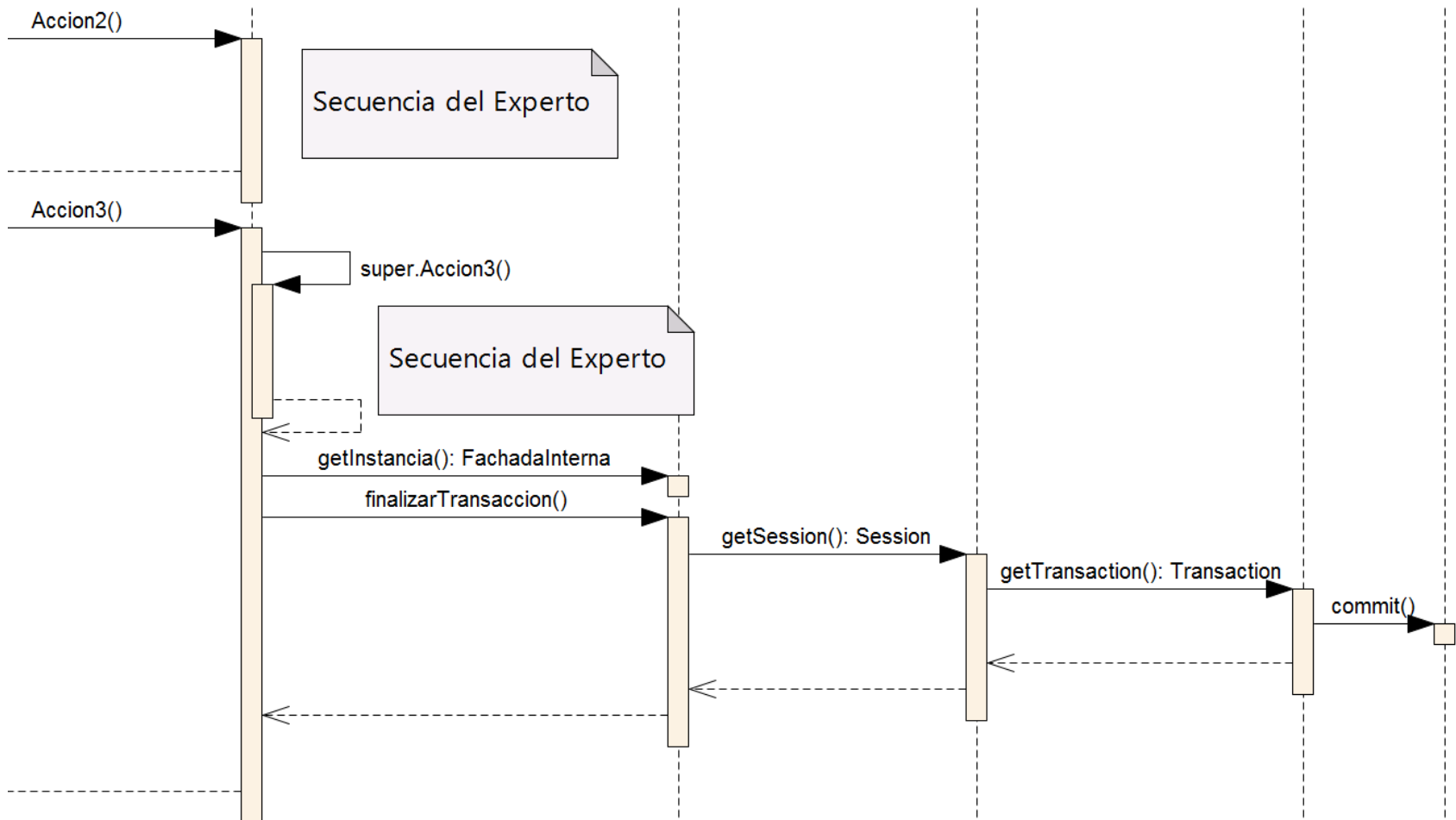
Transacciones – Decorador



Transacciones – Decorador



Transacciones – Decorador



Transacciones – Decorador

- Ninguno de nuestros expertos ha sufrido el impacto de agregar transacciones. Por lo tanto, el resultado es muy bueno.
- Sin embargo alguien sufre este nuevo cambio, y son justamente los controladores que son quienes acceden a los expertos.

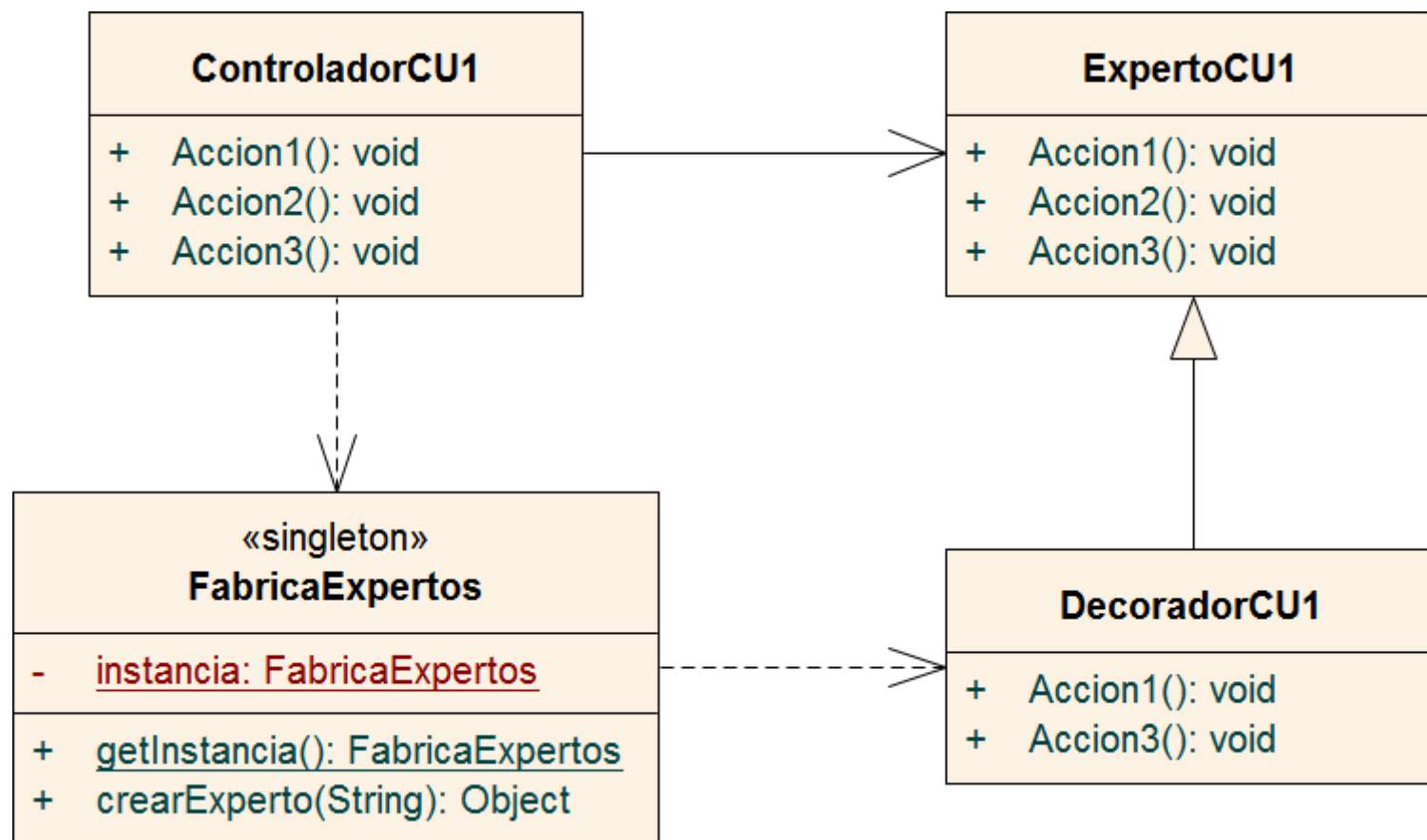
Transacciones – Decorador

- Anteriormente los controladores creaban instancias de los expertos para luego solicitarle la ejecución de diversas tareas.
- Ahora los controladores deberán crear instancias de los decoradores. Para evitarlo crearemos una fábrica accesible por los controladores responsable de crear las instancias de los expertos a medida que se solicite.

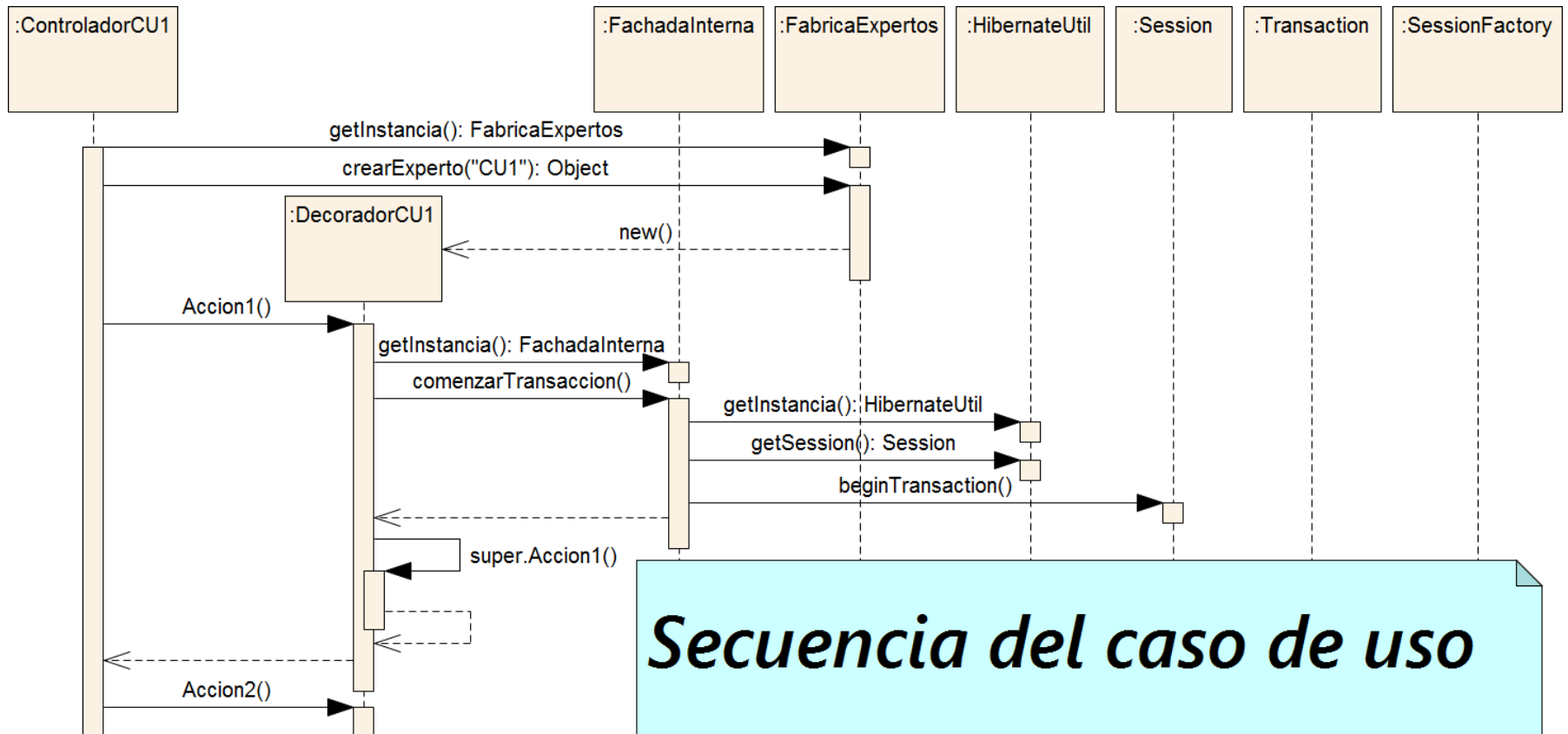
Transacciones – Decorador

```
public class ControladorCU1 {  
    ExpertoCU1 experto = (ExperoCU1) FabricaExpertos.getInstance().crearExperto("CU1");  
  
    public void Accion1() {  
        experto.Accion1();  
    }  
  
    //Resto del codigo del controlador  
}  
  
public class FabricaExpertos {  
    private static FabricaExpertos instancia = null;  
  
    public static FabricaExpertos getInstance() {  
        if(instancia == null)  
            instancia = new FabricaExpertos();  
        return instancia;  
    }  
  
    public Object crearExperto(String casoDeUso) {  
        switch(casoDeUso) {  
            case "CU1":  
                return new DecoradorCU1();  
            case "CU2":  
                return new DecoradorCU2();  
            default:  
                return null;  
        }  
    }  
}
```

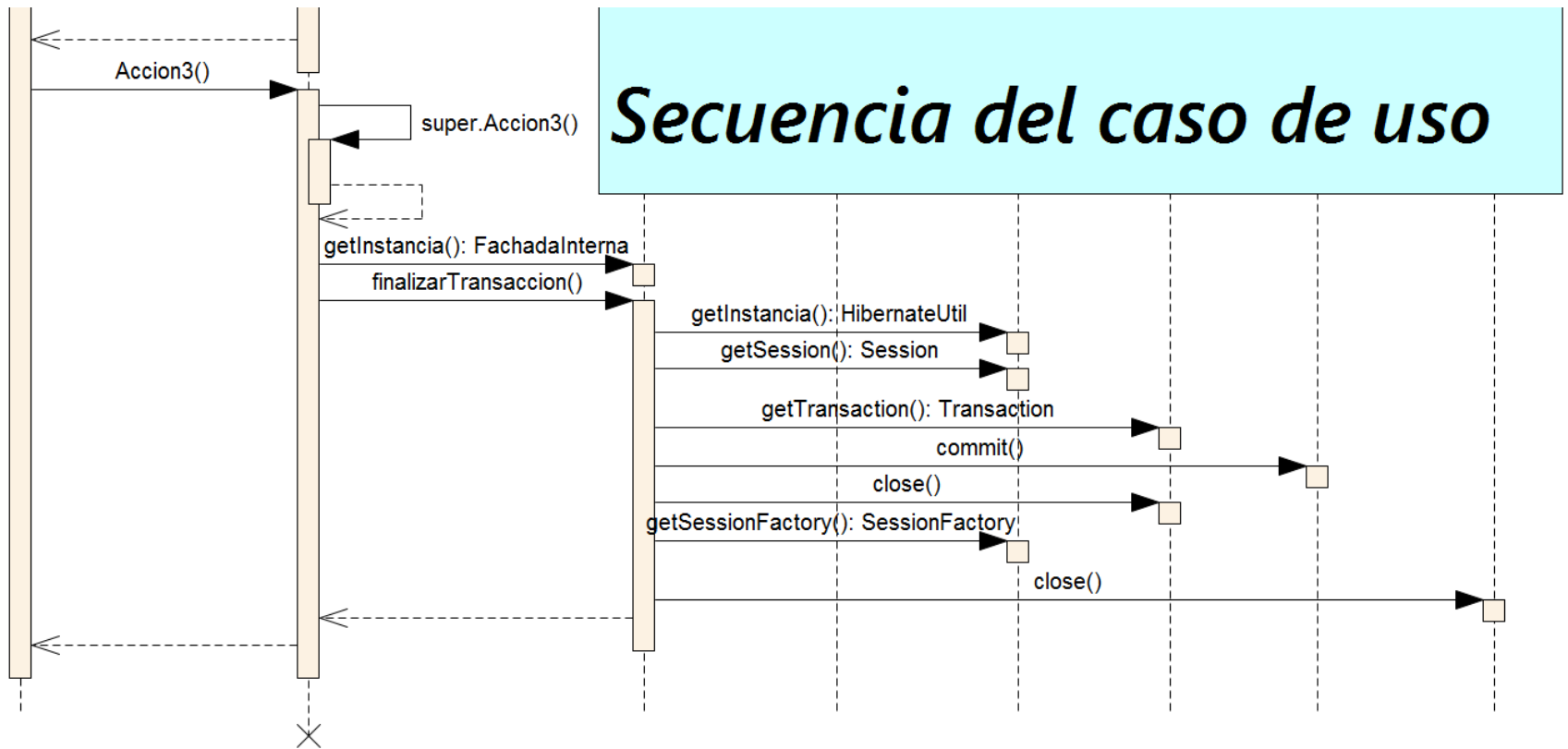
Transacciones – Decorador



Transacciones – Decorador



Transacciones – Decorador



Transacciones – Decorador

